

```

ask n-of N_shops patches
[ sprout-shops 1 [
  set shape "square"
  set colour grey
  set pcolor yellow
  set shop-coins starting-change
  set no-change 0
  set count-sales 0
]
]
end

to setup-shoppers
  create-shoppers N_shoppers [
    setxy random-xcor random-ycor
    set color blue
    set shape "dot"
    set coins 0
  ]
end

```

The modelling code runs for 500 ticks. Each shopper is asked to move forward a step and check if there is any shop in that location. If there is a shop, a sale is attempted. The value of each sale is selected at random by a number that ranges between 1 and the note value. If a shopper has enough coins, (s)he pays with the coins. If (s)he does not have enough coins, (s)he pays with a note and the shop returns the change if it has enough coins. The sale fails if that is not possible.

```

to go
  if ticks >= 500 [ stop ]
  ask shops [ set color grey ]
  ask shoppers [
    forward 1
    if any? shops-here [
      make-sale
    ]
  ]
  tick
end

to make-sale
  let value max (list 1 random note-value)
  ifelse coins >= value
  [ set coins coins - value
    ask shops-here
    [ set shop-coins shop-coins + value
      set color green
    ]
  ]
end

```

```

set count-sales count-sales + 1
set sale sale + value
]
[ ask shops-here
  [ ifelse shop-coins >= (note-value - value)
    [ set shop-coins shop-coins - (note-value - value)
      ask myself [ set coins coins + (note-value - value) ]
      set sale sale + value
      set count-sales count-sales + 1
      set color cyan
    ]
    [ set no-change no-change + 1
      set color red
    ]
  ]
]
end

```

During the running of the model, the interface shows the failure rate and the revenue per sale attempted.

Running the model

The objective is to code. But you can verify that some of our intuitive perceptions are reasonable. You may choose the number of coins in a note to be 20 (modelled on ₹ 2000/₹ 100). Let's suppose there are 100 shops and 500 shoppers, and the shops start out with change of a 100 coins each. We expect that the failure rate will be low. It stabilises at about 5 per cent with an average earning per visit of nine coins. Now let's suppose that the starting change of each shop is 20 coins. How badly would the sales be affected? In our model, the failure rate reaches over 60 per cent and the average earnings per visit drop to about 3.5 coins.

You can explore and play around with the model.

The keyword 'ask' is the critical construct which allows you to code for a specific breed of turtles or patches (the location/coordinates in the world).

The 'trickiest' code is the creation of N shops located randomly. In this case, you ask any N patches to 'sprout' a shop (ask n of N patches ...).

This is a very useful feature if you need to have only a random subset of turtles to do something.

The key takeaway is that surprisingly little code is needed to start exploring complex phenomena and understand the statistical implications of even simple interactions. 

By: Dr Anil Seth

The author has earned the right to do what interests him. You can find him online at <http://sethanil.com> and <http://sethanil.blogspot.com>, and reach him via email at anil@sethanil.com.